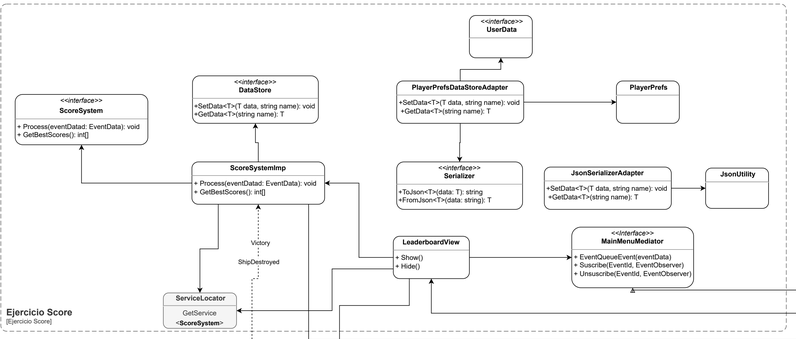
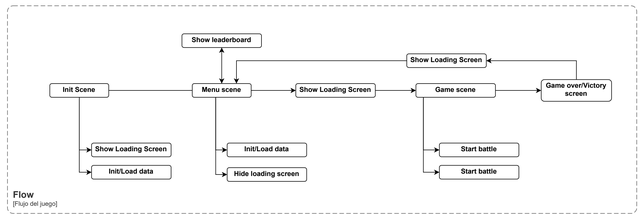
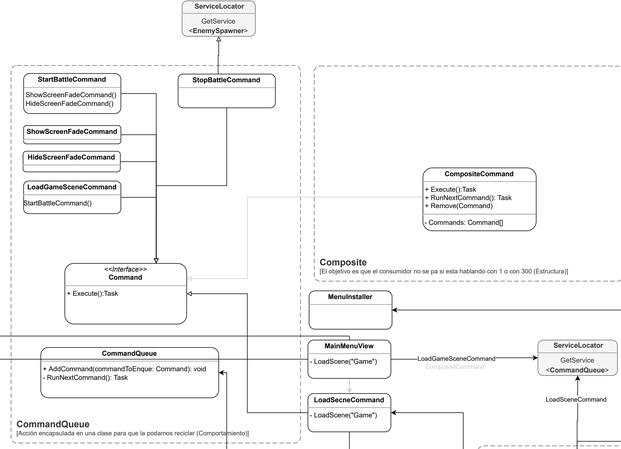
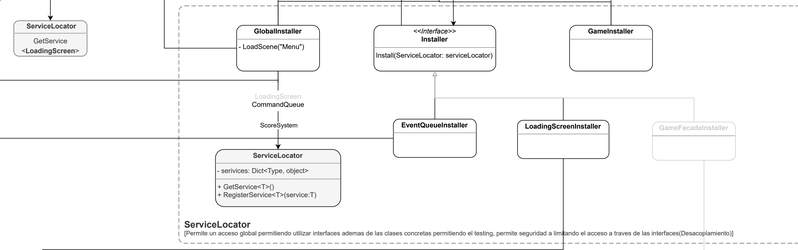

Ejercicio Score
[Ejercicio Score]
<<interface>>
ScoreSystem
+ Process(eventDatad: EventData): void
+ GetBestScores(): int[]
<<interface>>
DataStore
+SetData<T>(T data, string name): void
+GetData<T>(string name): T
<<interface>>
UserData
PlayerPrefsDataStoreAdapter
+SetData<T>(T data, string name): void
+GetData<T>(string name): T
PlayerPrefs
ScoreSystemImp
+ Process(eventDatad: EventData): void
+ GetBestScores(): int[]
<<interface>>
Serializer
+ToJson<T>(data: T): string
+FromJson<T>(data: string): T
JsonSerializerAdapter
+SetData<T>(T data, string name): void
+GetData<T>(string name): T
JsonUtility
Victory
ShipDestroyed
LeaderboardView
+ Show()
+ Hide()
<<interface>>
MainMenuMediator
+ EventQueueEvent(eventData)
+ Suscribe(EventId, EventObserver)
+ Unsuscribe(EventId, EventObserver)
ServiceLocator
GetService
<ScoreSystem>
ServiceLocator
GetService<EventQueue>
+Suscribe
ScoreEntryView

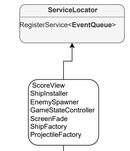

Flow
[Flujo del juego]
Show leaderboard
Init Scene
Menu scene
Show Loading Screen
Game scene
Game over/Victory screen
Show Loading Screen
Init/Load data
Init/Load data
Hide loading screen
Start battle
Start battle


CommandQueue
[Acción encapsulada en una clase para que la podamos reciclar (Comportamiento)]
ServiceLocator
GetService
<EnemySpawner>
StartBattleCommand
ShowScreenFadeCommand()
HideScreenFadeCommand()
StopBattleCommand
ShowScreenFadeCommand
HideScreenFadeCommand
LoadGameSceneCommand
StartBattleCommand()
<<interface>>
Command
+ Execute():Task
CommandQueue
+ AddCommand(commandToEnque: Command): void
- RunNextCommand(): Task
MainMenuView
- LoadScene("Game")
LoadGameSceneCommand
CompositeCommand
LoadSecneCommand
- LoadScene("Game")
ServiceLocator
GetService
<LoadingScreen>

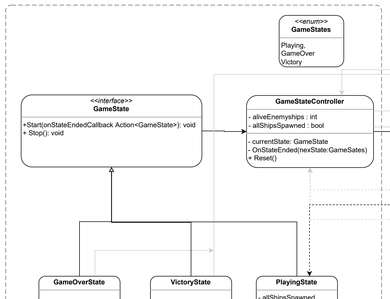
Composite
[El objetivo es que el consumidor no se pa si esta hablando con 1 o con 300 (Estructura)]
CompositeCommand
+ Execute():Task
+ RunNextCommand(): Task
+ Remove(Command)
- Commands: Command[]

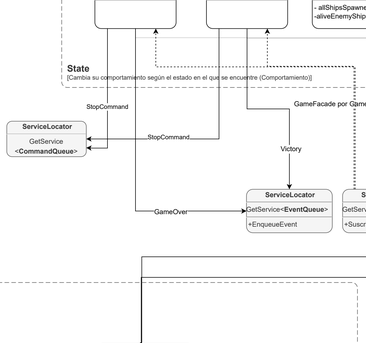
MenuInstaller
ServiceLocator
GetService
<CommandQueue>
LoadSceneCommand
GeneralInstaller
GlobalInstaller
- LoadScene("Menu")
<<Interface>>
Installer
Install(IServiceLocator: serviceLocator)
GameInstaller
CommandQueue
ScoreSystem
EventQueueInstaller
LoadingScreenInstaller
GameFacadeInstaller
ServiceLocator
- services: Dict<Type, object>
+ GetService<T>()
+ RegisterService<T>(service:T)


ServiceLocator
[Permite un acceso global permitiendo utilizar interfaces ademas de las clases concretas permitiendo el testing, permite seguridad a limitando el acceso a traves de las interfaces(Desacoplamiento)]


ServiceLocator
RegisterService<EventQueue>
ScoreView
ShipInstaller
EnemySpawner
GameStateController
ScreenFade
ShipFactory
ProjectileFactory

ServiceLocator
GetService<EventQueue>
+Suscribe
ScoreEntryView


State
[Cambia su comportamiento según el estado en el que se encuentre (Comportamiento)]
<<enum>>
GameStates
Playing,
GameOver
Victory
ServiceLocator
GetService
<GameFacade>
ServiceLocator
GetService
<CommandQueue>
<<interface>>
GameState
+Start(onStateEndedCallback Action<GameState>): void
+ Stop(): void
GameStateController
- aliveEnemyships : int
- allShipsSpawned : bool
- currentState: GameState
- OnStateEnded(nextState:GameSates)
+ Reset()
StopCommand
GameOver
Victory
ShipSpawned
AllShipsSpawned
GameOverState
VictoryState
PlayingState
- allShipsSpawned
-aliveEnemyShips
StopCommand
StopCommand
GameFacade por GameStateController
Victory
Victory
GameOver
ServiceLocator
GetService
<CommandQueue>
ServiceLocator
GetService<EventQueue>
+EnqueueEvent
ServiceLocator
GetService<EventQueue>
+Suscribe
ServiceLocator
GetService
<GameFacade>

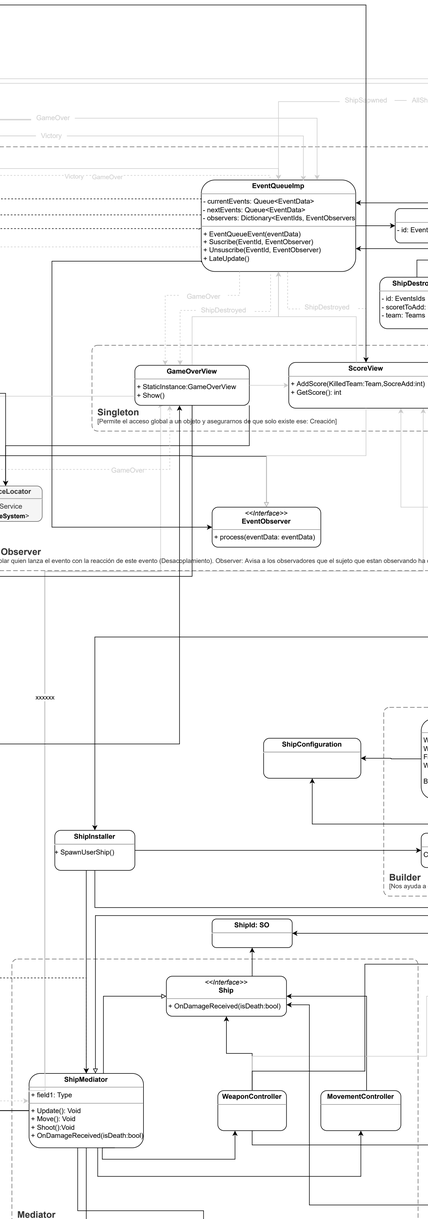

Mediator
[Desacoplar Componentes: Comportamiento]
LoadSceneCommand(IMenu)
ResumeGameCommand
PauseGameCommand
StopBattleCommand
StartBattleCommand
RestartBattleCommand
ServiceLocator
GetService
<CommandQueue>
Restart
ServiceLocator
GetService<EventQueue>
+EnqueueEvent
<<interface>>
EventObserver
Process(eventData: EventData)
<<interface>>
InGameMenuMediator
+OnBackToMenuPressed()
+OnRestartGamePressed()
+OnResumeGamePressed()
+OnPauseGamePressed()
InGameMenuView
+OnBackToMenuPressed()
+OnRestartGamePressed()
+OnResumeGamePressed()
+OnPauseGamePressed()
PauseView

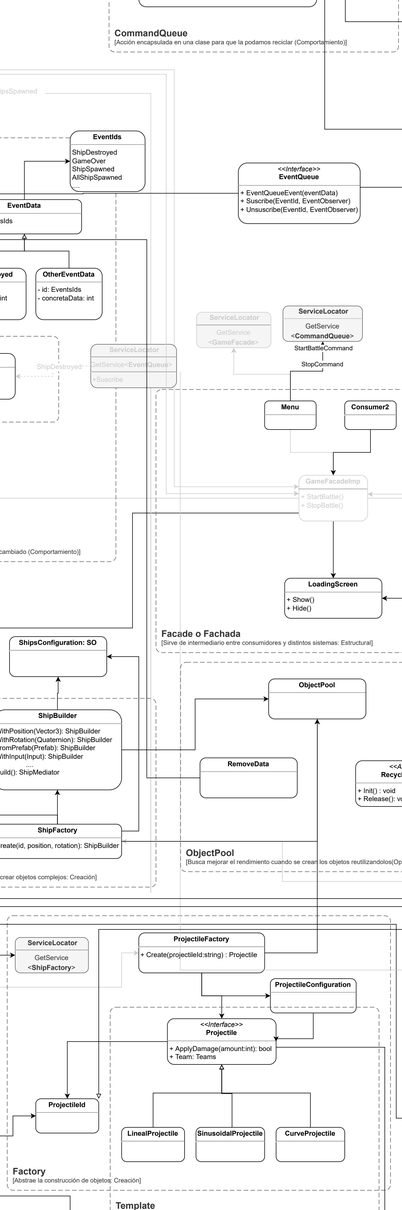

ResumeGameCommand
PauseGameCommand
RestartBattleCommand


ResumeGameCommand
PauseGameCommand
StopBattleCommand
StartBattleCommand
RestartBattleCommand
ServiceLocator
GetService
<CommandQueue>

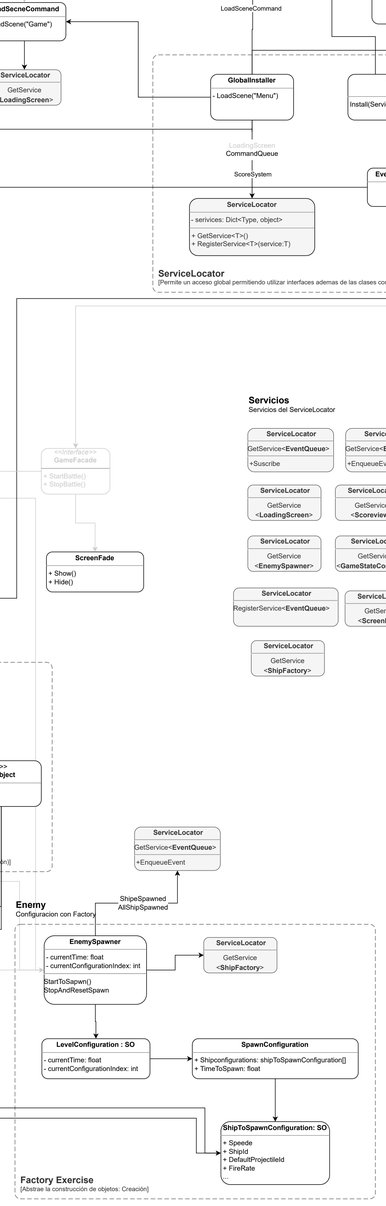
ServiceLocator
GetService<EventQueue>
+EnqueueEvent
ServiceLocator
GetService<EventQueue>
+Suscribe
GameOver
Victory
Restart
ShipDestroyed

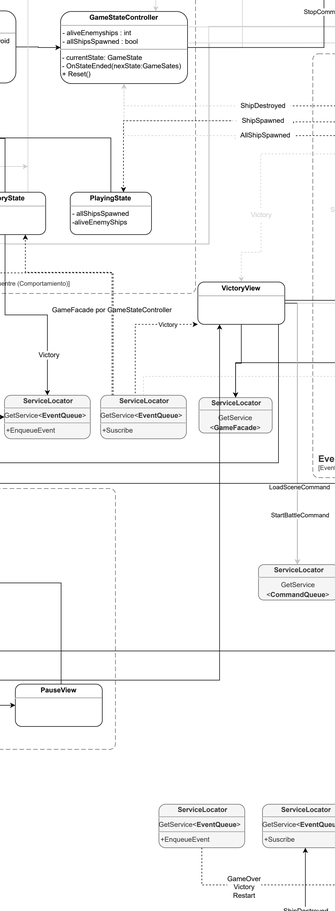
Strategy Exercise
[Aislar lógica de consumidores: Comportamiento]
<<interface>>
CheckDestroyLimits
+ IsInsideTheLimits(position: Vector3):bool
CheckBottomLimitsStrategy
+ IsInsideTheLimits(position: Vector3):bool
DoNotcheckDestroyLimitsStrategy
+ IsInsideTheLimits(position: Vector3):bool

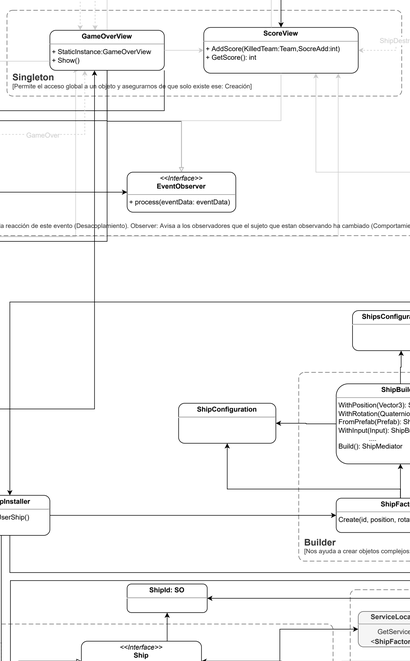

Mediator
[Desacoplar Componentes: Comportamiento]
ShipId: SO
<<Interface>>
Ship
+ OnDamageReceived(isDeath:bool)
ShipMediator
+ field1: Type
+ Update(): Void
+ Move(): Void
+ Shoot():Void
+ OnDamageReceived(isDeath:bool)
WeaponController
MovementController


Factory
[Abstrae la construcción de objetos: Creación]
ServiceLocator
GetService
<ShipFactory>
ProjectileFactory
+ Create(projectileId:string) : Projectile
ProjectileConfiguration
<<Interface>>
Projectile
+ ApplyDamage(amount:int): bool
+ Team: Teams
ProjectileId
LinealProjectile
SinusoidalProjectile
CurveProjectile

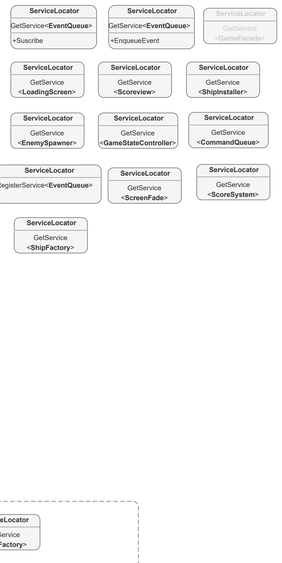
Template
[Define la base de un algoritmo permitiendo que los hijos lo sobrescriban :Comportamiento]

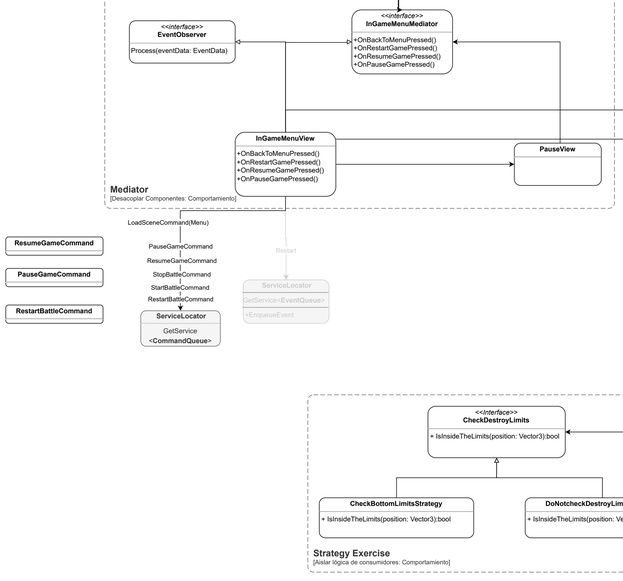

Enemy
Configuracion con Factory
ShipeSpawned
AllShipSpawned
EnemySpawner
- currentTime: float
- currentConfigurationIndex: int
StartToSapwn()
StopAndResetSpawn
ServiceLocator
GetService
<ShipFactory>
LevelConfiguration : SO
- currentTime: float
- currentConfigurationIndex: int
SpawnConfiguration
+ Shipconfigurations: shipToSpawnConfiguration[]
+ TimeToSpawn: float
ShipToSpawnConfiguration: SO
+ Speede
+ ShipId
+ DefaultProjectileId
+ FireRate
Factory Exercise
[Abstrae la construcción de objetos: Creación]

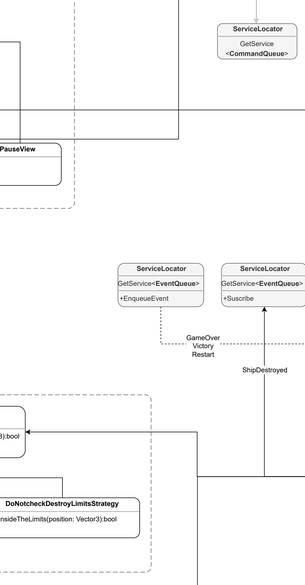

EventQueue y Observer
[EventQueue: Desacoplar quien lanza el evento con la reacción de este evento (Desacoplamiento). Observer: Avisa a los observadores que el sujeto que estan observando ha cambiado (Comportamiento)]
EventIds
ShipDestroyed
GameOver
ShipSpawned
AllShipSpawned
EventQueueImp
- currentEvents: Queue<EventData>
- nextEvents: Queue<EventData>
- observers: Dictionary<EventIds, EventObservers
+ EventQueueEvent(eventData)
+ Suscribe(EventId, EventObserver)
+ Unsuscribe(EventId, EventObserver)
+ LateUpdate()
EventData
- id: EventsIds
ShipDestroyed
- id: EventsIds
- scoreToAdd: int
- team: Teams
OtherEventData
- id: EventsIds
- concretaData: int
<<interface>>
EventQueue
+ EventQueueEvent(eventData)
+ Suscribe(EventId, EventObserver)
+ Unsuscribe(EventId, EventObserver)
ServiceLocator
- services: Dict<Type, object>
+ GetService<T>()
+ RegisterService<T>(service:T)


Singleton
[Permite el acceso global a un objeto y aseguramos de que solo existe ese: Creación]
GameOverView
+ StaticInstance:GameOverView
+ Show()
ScoresView
+ AddScore(KilledTeam:Team,ScoreAdd:int)
+ GetScore(): int
ServiceLocator
GetService<EventQueue>
+Suscribe
ShipDestroyed

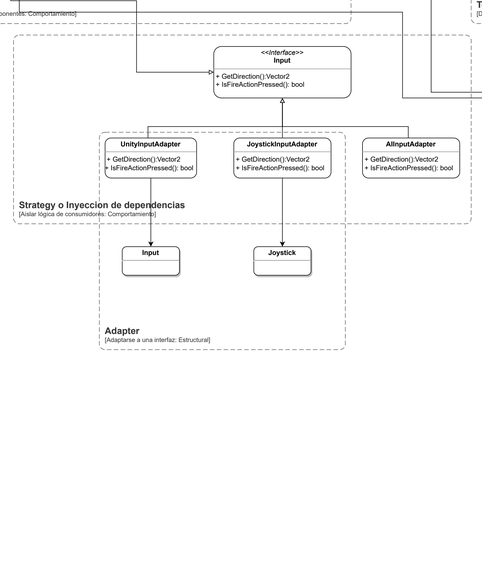
ServiceLocator
GetService
<GameFacade>
ServiceLocator
GetService
<CommandQueue>
StartBattleCommand
StopCommand
Menu
Consumer2
<<Interface>>
GameFacade
+ StartBattle()
+ StopBattle()
GameFacadeImp
+ StartBattle()
+ StopBattle()
LoadingScreen
+ Show()
+ Hide()
ScreenFade
+ Show()
+ Hide()
Facade o Fachada
[Sirve de intermediario entre consumidores y distintos sistemas: Estructural]


Servicios
Servicios del ServiceLocator
ServiceLocator
GetService<EventQueue>
+Suscribe
ServiceLocator
GetService<EventQueue>
+EnqueueEvent
ServiceLocator
GetService
<GameFacade>
ServiceLocator
GetService
<LoadingScreen>
ServiceLocator
GetService
<ScoreView>
ServiceLocator
GetService
<ShipInstaller>
ServiceLocator
GetService
<EnemySpawner>
ServiceLocator
GetService
<GameStateController>
ServiceLocator
GetService
<CommandQueue>
ServiceLocator
RegisterService<EventQueue>
ServiceLocator
GetService
<ScreenFade>
ServiceLocator
GetService
<ScoreSystem>
ServiceLocator
GetService
<ShipFactory>


Mediator
[Desacoplar Componentes: Comportamiento]
LoadSceneCommand(IMenu)
ResumeGameCommand
PauseGameCommand
StopBattleCommand
StartBattleCommand
RestartBattleCommand
ServiceLocator
GetService
<CommandQueue>
Restart
ServiceLocator
GetService<EventQueue>
+EnqueueEvent
<<interface>>
EventObserver
Process(eventData: EventData)
<<interface>>
InGameMenuMediator
+OnBackToMenuPressed()
+OnRestartGamePressed()
+OnResumeGamePressed()
+OnPauseGamePressed()
InGameMenuView
+OnBackToMenuPressed()
+OnRestartGamePressed()
+OnResumeGamePressed()
+OnPauseGamePressed()
PauseView


LoadSceneCommand
StartBattleCommand
ServiceLocator
GetService
<CommandQueue>
xxxxxx
ShipsConfiguration: SO
ShipConfiguration
ShipBuilder
WithPosition(Vector3): ShipBuilder
WithRotation(Quaternion): ShipBuilder
FromPrefab(Prefab): ShipBuilder
WithInput(Input): ShipBuilder
Build(): ShipMediator
ObjectPool
RemoveData
<<Abstract>>
RecyclableObject
+ Init() : void
+ Release(): void
ShipInstaller
+ SpawnUserShip()
ShipFactory
Create(id, position, rotation): ShipBuilder
Builder
[Nos ayuda a crear objetos complejos: Creación]
ObjectPool
[Busca mejorar el rendimiento cuando se crean los objetos reutilizandolos(Optimizacion)]


Strategy o Inyeccion de dependencias
[Aislar lógica de consumidores: Comportamiento]
<<interface>>
CheckLimits
+ ClampFinalPosition():Void
ViewportCheckLimits
+ ClampFinalPosition():Void
InitialPositionCheckLimits
+ ClampFinalPosition():Void


Strategy o Inyeccion de dependencias
[Aislar lógica de consumidores: Comportamiento]
<<interface>>
Input
+ GetDirection():Vector2
+ IsFireActionPressed(): bool
UnityInputAdapter
+ GetDirection():Vector2
+ IsFireActionPressed(): bool
JoystickInputAdapter
+ GetDirection():Vector2
+ IsFireActionPressed(): bool
AIInputAdapter
+ GetDirection():Vector2
+ IsFireActionPressed(): bool
Input
Joystick
Adapter
[Adaptarse a una interfaz: Estructural]


HealthController
+ ApplyDamage(amount:int): bool
+ event:OnDamageReceived(isDeath:bool)
+ Team: Teams
<<Interface>>
Damageable
+ ApplyDamage(amount:int): bool
+ Team: Teams